

FleCSI: Flexible Computational Science Infrastructure

Benjamin Bergen¹, Nick Moss², Irina Demeshko³, Davis Herring¹, Marc Charest⁴, Julien Loiseau¹, Navamita Ray¹, Jonathan Graham¹, Hartmut Kaiser⁵, Li-Ta Lo¹, Karen Tsai¹, Charles Ferenbaugh¹, Richard Berger¹, John Wohlbier⁶, Jonas Lippuner², Wei Wu³, Andrew Reisner¹, Scott Pakin¹, Brendan K. Krueger¹, Lukas Spies⁷, Sumathi Lakshmiranganatha¹, Max Ortner², Pascal Grosset¹, David Gunter², Maxim Moraru¹, Galen Shipman¹, Jiajia Waters¹, Scot A. Halverson³, Onur Çaylak², Peter Brady¹, Philipp V. F. Edelmann¹, Mason Delan², Brandon Keim⁸, Christopher M. Malone¹, Alex Villa⁹, Daniel Holladay¹, Dani Barrack², Nikunj Gupta¹⁰, Ondřej Čertík⁴, Robert Bird², Melissa Rasmussen¹¹, and Christoph Junghans¹

¹ Los Alamos National Laboratory, USA ² Independent researcher ³ NVIDIA, USA ⁴ Microsoft, USA ⁵ Louisiana State University, USA ⁶ Software Engineering Institute, USA ⁷ INRIA, France ⁸ University at Buffalo, USA ⁹ University of California, Merced, USA ¹⁰ Databricks, USA ¹¹ Stony Brook University, USA

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- Review [↗](#)
- Repository [↗](#)
- Archive [↗](#)

Editor: [Jed Brown](#) [↗](#) 

Reviewers:

- [@lindsayad](#)
- [@mprobson](#)

Submitted: 05 August 2025

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

FleCSI (Bergen et al., 2021) is a modern C++ framework designed to support the development of multiphysics simulations. It provides a task-based programming model that unifies shared- and distributed-memory programming. FleCSI provides high performance, flexibility, and portability across heterogeneous computing architectures.

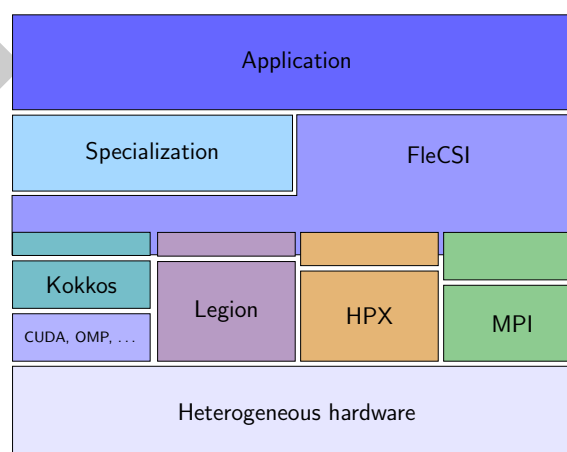


Figure 1: The FleCSI software ecosystem

Statement of need

FleCSI is designed to support the development of multiphysics simulations through a flexible, task-based programming model that enables performance portability across distributed, heterogeneous systems. It advances prior work by integrating dynamic task scheduling, data abstraction, and backend interoperability within a unified C++ framework. Compared to related systems like Uintah (Meng et al., 2012) and MPC (Pérache et al., 2008), FleCSI offers greater extensibility and finer runtime control, placing it at the intersection of portability, scalability, and modern software design for scientific computing.

Software description

FleCSI is designed to abstract away complexity while offering fine control for high-performance computing. The FleCSI runtime system manages initialization, execution, and shutdown. As presented in Figure 1, the FleCSI runtime supports backends such as Legion (Bauer et al., 2012), HPX (Kaiser et al., 2009, 2020), MPI (Message Passing Interface Forum, 2025), and Kokkos (Edwards et al., 2014), enabling code to remain portable across a variety of systems without manually handling the execution environment.

FleCSI's programming model is based on a hierarchy of parallelism: sequential, task-parallel, and data-parallel. The relationships among these is illustrated in Figure 2:

- **Control points (CP)** define an application's sequential backbone.
- **Actions (A)** specify a directed acyclic graph of high-level operations and their dependencies.
- **Tasks** are functions that operate on data distributed across address spaces.
- **Point tasks (PT)** are individual instances of a task that operate on a local fragment of a distributed data structure.
- **Kernels (K)** process a block of local data in a data-parallel fashion on a CPU or GPU.

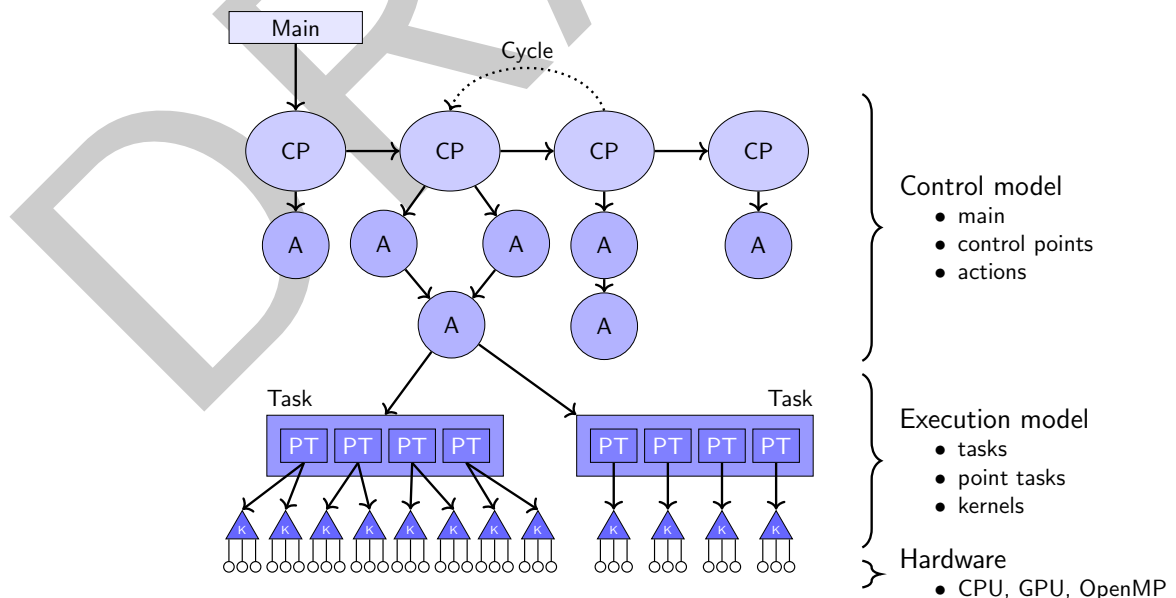


Figure 2: FleCSI control and execution models.

FleCSI's *control model* comprises control points and actions and determines what work is

performed and in what order. FleCSI's *execution model*, comprising tasks, point tasks, and kernels, governs where and how that work actually runs. FleCSI's *data model*, not shown in Figure 2, governs how data are distributed and accessed.

Control model

Control points specify an application's sequential control flow and can include conditional branches. For example, one control point may represent "initialization", another "repetition until convergence", and a third "finalization".

Control points provide hooks for a directed acyclic graph (DAG) of *actions* to be attached. An action is a sequential function that defines an application's core numerics or physics routines such as "hydrodynamics", "radiative diffusion", or "reaction network". A new action can be incorporated into an application by specifying its direct dependents and dependencies (control points or other actions). For example, if an existing application defines a "solver iteration" action, a new developer later can create a "visualization" action and insert it after "solver iteration" in the DAG without having to modify any other code or interfaces. If other actions depend on "solver iteration", these will run concurrently with "visualization". By walking the DAG in topological order, FleCSI ensures a valid program execution.

Execution model

Actions spawn *tasks*, which are functions that are distributed within and across the nodes of the compute cluster and that complete asynchronously. In a computational-science application, a task typically represents updates to a data structure, such as to perform mesh operations (e.g., relaxation). Because there exist run-time costs in launching tasks and moving data across a large-scale, hybrid CPU/GPU cluster, task granularity should be large enough to amortize these costs. A rule of thumb is for tasks to execute in no less than about 10 ms to keep the relative overhead manageable.

A task declaration includes the *fields* of a distributed data structure that it will access (defined on, say, the cells, edges, and vertices of an unstructured mesh) and the access rights it requires on each field: read only, write only, or read/write. Tasks are run concurrently according to field data dependencies. For example, if task A reads x and writes y , task B reads x and writes z , and task C reads y and writes w , then the FleCSI runtime will execute tasks A and B concurrently but require that task A finish before task C can start.

Distributed parallelism is achieved by decomposing field domains into *colors*. For any one color, memory for a field is allocated contiguously, but colors do not need to map 1:1 to processes. Rather, the application chooses an appropriate number of colors for each task launch. If colors outnumber processes then some processes simply handle more than one color. Each color is handled by exactly one *point task*—an individual instance of a task. All point tasks are executed on CPUs, but the data for readable fields are preloaded into a specified memory space (CPU NUMA domain or GPU device memory), and the data for writable fields are automatically communicated to dependent tasks.

Point tasks process their data by launching data-parallel *kernels* that operate on the memory space in which the field data was placed. In a computational-science application, these typically perform element updates such as incrementing position, momentum, energy, etc. Kernels can execute in parallel on GPUs, in parallel on CPUs (using OpenMP threads), or serially on CPUs. Kernel code is portable across these three forms of execution; no code modifications are needed to dispatch a kernel to a CPU versus a GPU. This is because FleCSI arranges for a kernel's data to be available locally before the kernel is launched—in either a CPU or GPU execution space—and because FleCSI provides `forall` and `reduceall` constructs that operate consistently across execution spaces, supporting data-parallel code execution. In cases where these constructs are too limiting, FleCSI supports *task variants*, whereby a program can provide different implementations for different execution spaces (e.g., using arbitrary Kokkos

97 calls in the GPU execution space and explicit OpenMP pragmas in the OpenMP execution
98 space) and let FleCSI select the appropriate variant at run time.

99 Data model

100 FleCSI provides several topology types—skeletons of distributed data structures—that
101 applications use to represent physical quantities and their relationships:

- 102 ■ `topo::unstructured` supports graph-based meshes and is suitable for finite element or
103 finite volume methods.
- 104 ■ `topo::narray` provides structured n -dimensional grids with support for boundary
105 conditions and periodicity, making it ideal for Eulerian hydrodynamics.
- 106 ■ `topo::ntree` organizes data in a hashed tree structure that enables fast neighbor searches
107 and is appropriate for particle-based simulations and adaptive mesh refinement.

108 Although topology data are distributed, all communication and synchronization is implicit and
109 is based on the access rights associated with each field. (See [Execution model](#) above.) Fields
110 can be defined with several layouts such as dense (arrays), ragged (vectors), sparse (maps), or
111 particle (buffers).

112 Acknowledgments

113 The FleCSI project is supported by the U.S. Department of Energy through Los Alamos
114 National Laboratory (LANL). Los Alamos National Laboratory is operated by Triad National
115 Security, LLC, for the National Nuclear Security Administration of the U.S. Department of
116 Energy (contract no. 89233218CNA000001). This paper has been assigned a Los Alamos
117 Unlimited Release number of LA-UR-25-25479.

118 The work reported in this paper would not have been possible without close collaborations
119 with the Legion and HPX teams and LANL's Ristra project, FleCSI's initial “customer”.

120 References

- 121 Bauer, M., Treichler, S., Slaughter, E., & Aiken, A. (2012). Legion: Expressing locality and
122 independence with logical regions. *SC'12: Proceedings of the International Conference on*
123 *High Performance Computing, Networking, Storage and Analysis*, 1–11. <https://doi.org/10.1109/SC.2012.71>
- 124
- 125 Bergen, B., Demeshko, I., Ferenbaugh, C., Herring, D., Lo, L.-T., Loiseau, J., Ray, N., &
126 Reisner, A. (2021). FleCSI 2.0: The flexible computational science infrastructure project.
127 *European Conference on Parallel Processing*, 480–495. [https://doi.org/10.1007/978-3-031-](https://doi.org/10.1007/978-3-031-06156-1_38)
128 [06156-1_38](https://doi.org/10.1007/978-3-031-06156-1_38)
- 129 Edwards, H. C., Trott, C. R., & Sunderland, D. (2014). Kokkos: Enabling manycore
130 performance portability through polymorphic memory access patterns. *Journal of Parallel*
131 *and Distributed Computing*, 74(12), 3202–3216. [https://doi.org/10.1016/j.jpdc.2014.07.](https://doi.org/10.1016/j.jpdc.2014.07.003)
132 [003](https://doi.org/10.1016/j.jpdc.2014.07.003)
- 133 Kaiser, H., Brodowicz, M., & Sterling, T. (2009). ParalleX: An advanced parallel execution
134 model for scaling-impaired applications. *2009 International Conference on Parallel*
135 *Processing Workshops*, 394–401. <https://doi.org/10.1109/icppw.2009.14>
- 136 Kaiser, H., Diehl, P., Lemoine, A. S., Lelbach, B. A., Amini, P., Berge, A., Biddiscombe, J.,
137 Brandt, S. R., Gupta, N., Heller, T., Huck, K., Khatami, Z., Kheirkhahan, A., Reverdell,
138 A., Shirzad, S., Simberg, M., Wagle, B., Wei, W., & Zhang, T. (2020). HPX—the C++
139 standard library for parallelism and concurrency. *Journal of Open Source Software*, 5(53),
140 2352. <https://doi.org/10.21105/joss.02352>

- 141 Meng, Q., Humphrey, A., & Berzins, M. (2012). The Uintah framework: A unified
142 heterogeneous task scheduling and runtime system. *2012 SC Companion: High*
143 *Performance Computing, Networking, Storage and Analysis (SCC)*, 2441–2448.
144 <https://doi.org/10.1109/SCC.2012.6674233>
- 145 Message Passing Interface Forum. (2025). *MPI: A message-passing interface standard version*
146 *5.0*. <https://www.mpi-forum.org/docs/mpi-5.0/mpi50-report.pdf>
- 147 Pérache, M., Jourden, H., & Namyst, R. (2008). MPC: A unified parallel runtime for
148 clusters of NUMA machines. *Euro-Par 2008 – Parallel Processing*, 5168, 78–88. https://doi.org/10.1007/978-3-540-85451-7_9
149

DRAFT